The Journal of Open Source Software

NFNull: A Python library for tail integral estimation via normalizing flows

Kipper Fletez-Brant[1¶], **Jason Xu**[2], **and Tushar Bhangale**[1]

1 Department of Human Genetics, Genentech, Inc., United States **2** Department of Biostatistics, University of California Los Angeles, United States ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Tail integrals—probabilities of the form $P(X > x)$ and conditional expectations such as $\mathbb{E}[X \mid X > x]$—arise throughout quantitative analysis. In hypothesis testing, p-values are tail integrals under a null distribution. When the target distribution lacks a closed form, these quantities are typically estimated via Monte Carlo (MC) methods: given B samples $\{X^{(1)}, \dots, X^{(B)}\}$, the tail probability is approximated as the fraction of samples exceeding the query threshold. This is simple and broadly applicable, but the empirical distribution yields exactly zero for any threshold unrepresented in the sample, and precision in the tails requires large B.

`NFNull` provides a Python interface for fitting normalizing flows (NFs) to Monte Carlo samples and using the learned distribution for tail integral estimation. NFs are neural-network-based density estimators that learn bijective mappings between data and a tractable base distribution (Papamakarios et al., 2021; Tabak & Turner, 2013), enabling both density evaluation and efficient sampling. Given B samples from a target distribution f_X, `NFNull` trains an NF—by default a neural spline flow (NSF) (Durkan et al., 2019) with rational-quadratic splines (Delbourgo & Gregory, 1982)—and then draws a large number of samples n_s from the fitted flow to estimate tail probabilities:

$$P(X > x) \approx \frac{1}{n_s} \sum_{j=1}^{n_s} \mathbf{1}(T(u_j) > x), \quad u_j \sim \mathcal{N}(0, 1).$$

The trained flow is a reusable surrogate: once fitted, it can answer tail queries at arbitrary thresholds without re-running the underlying simulation, supporting amortized inference (Gershman & Goodman, 2014; Kingma & Welling, 2014) across many downstream tail queries.

Statement of need

Many scientific and statistical workflows require tail probability estimation from simulation- or bootstrap-generated samples: permutation p-values in genomics, portfolio risk metrics derived from return scenarios, or test statistics under complex null models. The primary audience is quantitative researchers in genomics, computational biology, finance, and related fields who must estimate tail probabilities from simulation or bootstrap output and need to query the fitted distribution at thresholds beyond the observed sample range or across many query points. Standard approaches include direct MC estimation and parametric density estimators such as Gaussian mixture models (GMMs) (Titterington et al., 1985) or kernel density estimation (Parzen, 1962; Rosenblatt, 1956). MC estimators are unbiased but return exactly zero for thresholds not represented in the training sample—a fundamental limitation

when tail probabilities below $1/B$ are of interest. Parametric methods require hyperparameter selection (bandwidth, number of components) that can strongly influence tail behavior, and may impose structural assumptions that misspecify the underlying distribution.

Normalizing flows are universal density approximators (Papamakarios et al., 2017, 2021) that learn an end-to-end mapping from data via gradient descent, without requiring explicit parametric structure. No existing Python package provides a focused, high-level interface for our target workflow: fitting a normalizing flow to simulation output and exposing it as a reusable CDF and tail-probability estimator. `NFNull` fills this gap, wrapping the `zuko` normalizing flow library (The Probabilists, 2024) in an API designed for the tail-estimation use case. The library is specifically aimed at settings where (i) only a finite sample from the target distribution is available, (ii) tail queries at thresholds beyond the observed sample range are needed, and (iii) the fitted model will be queried repeatedly or across many query points (e.g., multiple portfolio allocations).

State of the field

Several existing Python packages address aspects of density estimation and tail probability computation, but none is designed for the specific workflow `NFNull` targets. `scipy.stats` (Virtanen et al., 2020) provides a large catalog of parametric distributions with analytic CDFs and tail probability methods, but requires the user to specify the distribution family in advance—it cannot fit an arbitrary simulation output. Kernel density estimators in `scikit-learn` (Pedregosa et al., 2011) and `statsmodels` produce smooth densities from data with computable CDFs, but their tail behavior is governed entirely by bandwidth selection: standard heuristics such as Silverman's rule minimize integrated squared error of the density globally, not accuracy at extreme quantiles. With Gaussian kernels—the most common choice—the estimated tails always decay as a Gaussian regardless of the true tail shape. Gaussian mixture models via `scikit-learn` require explicit component count selection and can be severely misspecified on nonlinear structure: in our multivariate simulation studies, the GMM KL divergence on the banana-shaped exemplar at $d = 5$ reaches approximately 2×10^{10}, reflecting total density misspecification where `NFNull` remains tractable (KL ≈ 26).

Normalizing flow libraries—including `zuko` (The Probabilists, 2024), `nflows`, and `normflows`—provide general-purpose flow architectures supporting density evaluation and sampling, but do not expose a CDF or tail-probability API. Users must implement their own Monte Carlo CDF estimator, select an appropriate architecture, manage data preprocessing, and handle reuse of flow samples across multiple query thresholds.

The case for a new library rather than a contribution to `zuko` rests on scope: `zuko` is a computational toolkit for flow architectures, not a statistical inference or tail estimation package. The design choices specific to `NFNull`—the default of $n_s = 10^6$ Monte Carlo samples for CDF estimation, the selection of NSF with coupling as the default based on empirical hyperparameter sensitivity analysis, conditional density support for grouped null distributions, and the `p_value()` and `get_cdf()` user-facing API—are outside the scope of a backend library and constitute the library's distinct scholarly contribution.

Software design

`NFNull` is available on GitHub at https://github.com/Genentech/nfnull and installable via `pip install nfnull`. Full documentation and worked examples are in the repository README; a test suite covering all core functionality is in `tests/`.

The central design challenge is that CDF estimation from a normalizing flow requires drawing large numbers of samples from the fitted distribution. While the CDF of a flow-transformed variable has the analytic form $F_X(x) = \Phi(T^{-1}(x))$ for a Gaussian-base flow, inverting the

learned neural mapping T at arbitrary query points is not tractable in closed form (Sastry et al., 2022). We therefore estimate the CDF by drawing n_s samples from the fitted flow and computing the empirical CDF over those samples. With the default $n_s = 10^6$, the Monte Carlo standard error on a tail probability of 10^{-4} is approximately 10^{-5}, which is sufficient for most applications, and the same sample set answers queries at any number of thresholds.

This sampling-intensive approach places a hard constraint on architecture choice. Masked autoregressive flows (Papamakarios et al., 2017) compute the inverse T^{-1} efficiently but require $\mathcal{O}(K \cdot d)$ sequential operations to draw a single sample, making large-scale sampling impractical at $n_s = 10^6$. We therefore restrict `NFNull` to architectures where both T and T^{-1} have analytic expressions and can be evaluated in parallel: neural spline flows (NSF) with rational-quadratic splines and coupling layers (Delbourgo & Gregory, 1982; Dinh et al., 2015; Durkan et al., 2019), and Bernstein polynomial flows (BPF) (Sick et al., 2021).

Between these two, a hyperparameter sensitivity study—500 training samples, five replicates per configuration, evaluated by log-likelihood under the data-generating process, two-sample KS test, and KL divergence—showed that NSFs have substantially lower variation across configurations than BPFs, particularly on multimodal exemplars where BPFs exhibited elevated KL divergence. NSF performance was largely insensitive to the number of bins and coupling transforms within the ranges we tested (see Validation); BPF performance depended more strongly on polynomial degree and hidden layer size. We therefore adopt NSF with rational-quadratic splines as the default architecture. BPF and Student-t-base variants (TNSF) remain available for users with heavier-tailed targets.

```python
from nfnull import NFNull
nf = NFNull(samples)   # samples: array from the target distribution
nf.fit_pdf()
p = nf.p_value(x=3.5) # P(X > 3.5); uses n_s = 1e6 flow samples by default
```

`NFNull` is built on `zuko` (The Probabilists, 2024) for flow architectures and `PyTorch` for optimization. GPU acceleration is available via `.to(device)`, conditional density estimation is supported via a `context` argument, and the library is callable from R via `reticulate` (R Core Team, 2023).

Validation

`NFNull` is validated through simulation studies on univariate and multivariate distributions. For three univariate exemplars (standard Gaussian, trimodal mixture, and skewed mixture), the fitted NSF closely matches both the PDF and CDF of the data-generating process using 500 training samples, and provides nonzero tail probability estimates at extermal values. Table 1 and Table 2 illustrate this: at a query point $x = 4.9$ for the trimodal exemplar, the NSF estimate (8.4×10^{-7}) and the analytic value (8×10^{-43}) both correctly identify the event as extremely unlikely (we note the discrepancy in extremity is due to Monte Carlo sampling, in that achieving a p-value on the order of 10^{-43} requires sampling 10^{43} Monte Carlo draws).

Table 1: Exemplar distribution definitions.

Exemplar	Distribution	Parameters
Normal	$\mathcal{N}(\mu, \sigma^2)$	$\mu = 0$, $\sigma = 1$
Trimodal	$\sum_{k=1}^{3} \pi_k \mathcal{N}(\mu_k, \sigma^2)$	$\boldsymbol{\pi} = (0.3, 0.3, 0.4)$, $\boldsymbol{\mu} = (-1, 0, 1.5)$, $\sigma = 0.25$
Skew	$\pi_1 \mathrm{Exp}(\lambda) + \pi_2 U(a_1, b_1) + \pi_3 U(a_2, b_2)$	$\boldsymbol{\pi} = (0.86, 0.10, 0.04)$, $\lambda = 1.5$, $(a_1, b_1) = (0, 11)$, $(a_2, b_2) = (9, 10)$

DRAFT

Table 2: Tail probability estimates $P(X > x)$ for each exemplar, computed analytically, and from the fitted NSF (mean and 95% CI over 3 independent training runs).

Exemplar	x	Analytic	NSF (mean)	95% CI lower	95% CI upper
Normal	4.9	4.8e-7	6.2e-6	0	1.8e-5
Trimodal	4.9	8e-43	2.0e-7	4.0e-9	4.0e-7
Skew	9.6	4.2e-2	2.0e-2	9.2e-4	4.0e-2

Figure 1 shows the fitted densities and CDFs for each exemplar visually.

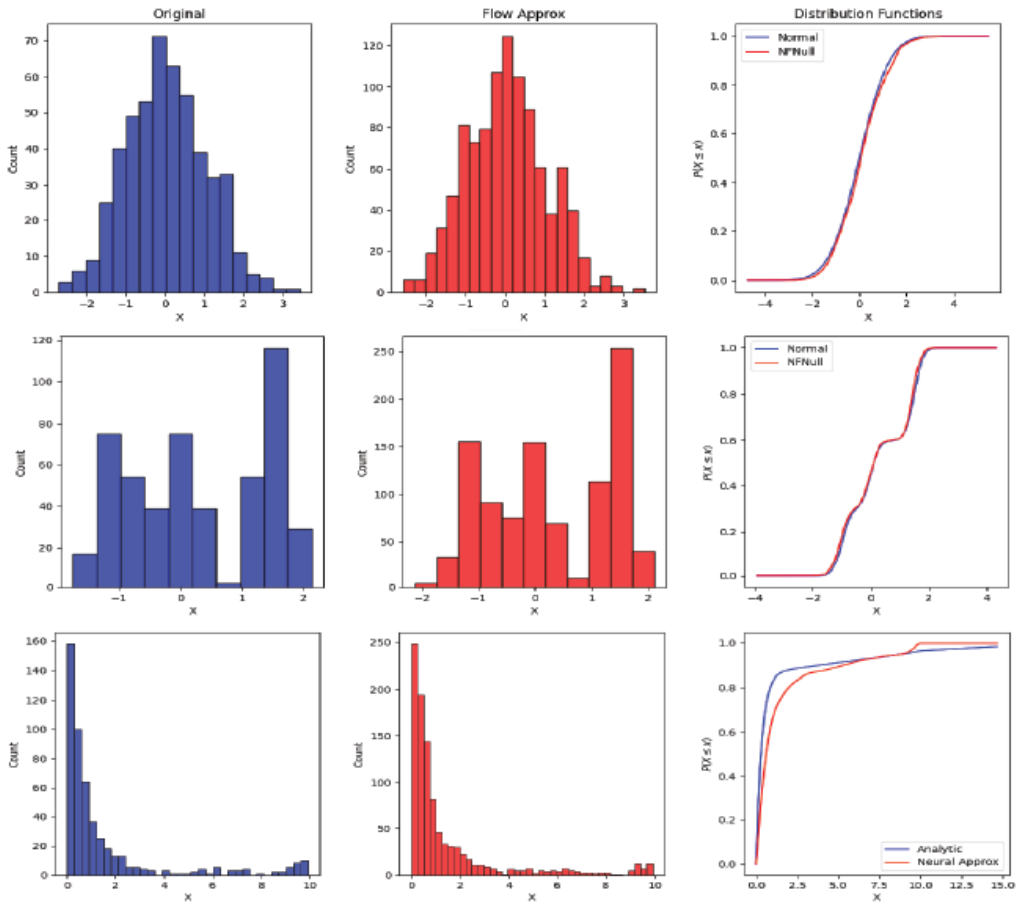

Figure 1: Normalizing flows provide flexible density approximations. Rows show exemplar datasets: standard Gaussian (top), trimodal (center), and skew (bottom). Columns show original training data (left), 1,000 draws from the fitted NSF (center), and the analytic CDF with the flow-estimated CDF overlaid (right). All exemplars were fit using an NSF with 12 bins, 2 single-layer MLPs with 64 hidden nodes, and coupling, on 500 training samples.

Figure 2 summarizes the hyperparameter sensitivity study across all three model classes. Each box aggregates results over hyperparameter configurations, with each configuration evaluated on five independent samples of 500 points. NSFs show substantially lower variation in log-likelihood, KS test, and KL divergence across configurations compared to BPFs and GMMs. BPF performance varies most on the trimodal exemplar, consistent with its sensitivity to polynomial degree on multimodal distributions. GMM performance depends on the number of components selected.

The Journal of Open Source Software

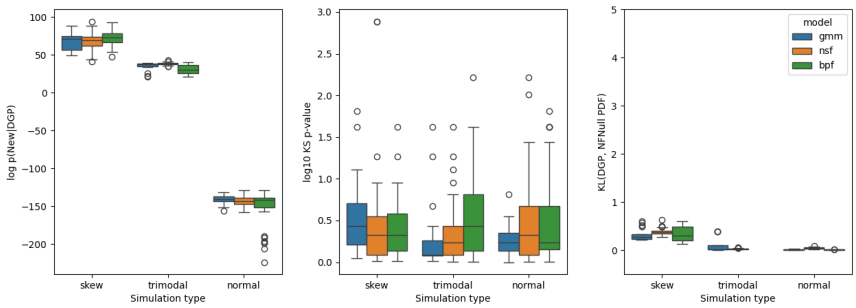

Figure 2: Quantitative evaluation across models and hyperparameters. Each box aggregates results over hyperparameter configurations, with each configuration evaluated on 5 independent samples of 500 points. Left: log-likelihood of model samples under the data-generating process. Center: $-\log_{10} p$-value from two-sample KS tests comparing 100 model samples to 100 new samples from the DGP. Right: KL divergence between DGP and model densities (BPF results on the trimodal exemplar omitted for scale). Within this design, NSFs show relatively stable performance across hyperparameter choices.

In multivariate settings ($d \in \{2, 5\}$) we evaluate six exemplar distributions: a Gaussian mixture model (Titterington et al., 1985), Clayton (Clayton, 1978; Nelsen, 2006) and Gumbel (Gumbel, 1960; Nelsen, 2006) copulas with asymmetric tail dependence, a multimodal hypercube mixture (McLachlan & Peel, 2000), a nonlinear banana-shaped distribution (Haario et al., 1999, 2001), and a t-copula with symmetric tail dependence (Demarta & McNeil, 2005).

Figure 3 compares KL divergence between the data-generating process and each model at $d = 2$ (top) and $d = 5$ (bottom), with the y-axis capped at 30 to keep the panels on a comparable scale. At $d = 2$, all three methods achieve low KL divergence across exemplars, with the largest values concentrated on Clayton ($\theta = 2$). At $d = 5$, the Banana distribution is the dominant failure mode: all methods show elevated KL divergence there, but the GMM bar is clipped at 30 (true value $\approx 2 \times 10^{10}$), reflecting severe density misspecification on nonlinear structure. NSF and TNSF reduce this failure substantially (KL ≈ 26 and ≈ 19, respectively), though both remain elevated, underscoring that empirical validation of tail fidelity remains important for any learned surrogate distribution.

DRAFT

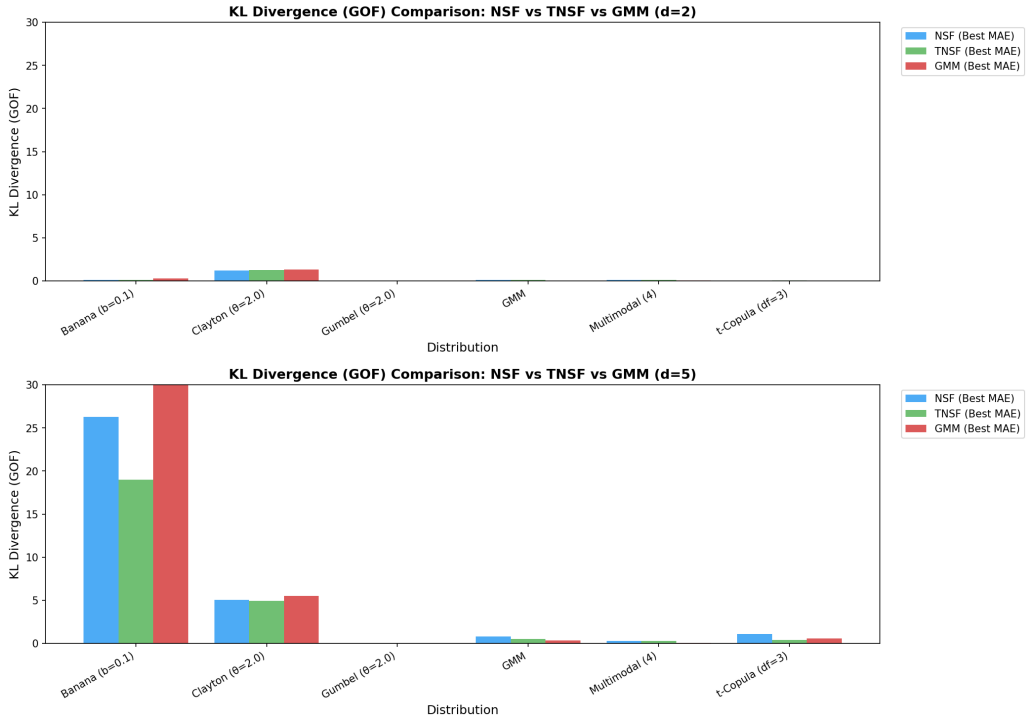

Figure 3: KL divergence between the data-generating process and fitted model at $d = 2$ (top) and $d = 5$ (bottom). The y-axis is capped at 30 across both panels for comparability; the true GMM KL on the Banana exemplar at $d = 5$ is approximately 2×10^{10}. At $d = 2$ all methods perform well; at $d = 5$ the Banana distribution is the primary challenge for all model classes.

AI usage disclosure

Anthropic's Claude Opus 4.5 was used in updates and refactoring of `NFNull`, in implementation of multivariate experiment scripts, and in drafting portions of this paper. Quality and correctness of AI-generated code contributions were verified through author code review and through the automated test suite in `tests/`, which covers univariate and multivariate density fitting, tail probability calibration across NSF and TNSF architectures, conditional density estimation, and batched context operations. The authors take full responsibility for all artifacts from this research, both code and paper.

Acknowledgements

The authors acknowledge William F. Forrest for helpful discussions.

K.F.-B. and T.B. are shareholders of Roche/Genentech stock. J.X. declares no competing financial interests.

References

Clayton, D. G. (1978). A model for association in bivariate life tables and its application in epidemiological studies of familial tendency in chronic disease incidence. *Biometrika*, *65*(1), 141–151.

Delbourgo, R., & Gregory, J. A. (1982). Rational quadratic spline interpolation to monotonic data. *IMA Journal of Numerical Analysis*.

DRAFT

Demarta, S., & McNeil, A. J. (2005). The t copula and related copulas. *International Statistical Review*, *73*(1), 111–129.

Dinh, L., Krueger, D., & Bengio, Y. (2015). NICE: Non-linear independent components estimation. *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Workshop Track Proceedings*. http://arxiv.org/abs/1410.8516

Durkan, C., Bekasov, A., Murray, I., & Papamakarios, G. (2019). Neural spline flows. In *Proceedings of the 33rd international conference on neural information processing systems*. Curran Associates Inc.

Gershman, S. J., & Goodman, N. D. (2014). Amortized inference in probabilistic reasoning. *Proceedings of the 36th Annual Conference of the Cognitive Science Society*.

Gumbel, E. J. (1960). Bivariate exponential distributions. *Journal of the American Statistical Association*, *55*(292), 698–707.

Haario, H., Saksman, E., & Tamminen, J. (1999). Adaptive proposal distribution for random walk metropolis algorithm. *Comput. Stat.*, *14*(3), 375–395. https://doi.org/10.1007/s001800050022

Haario, H., Saksman, E., & Tamminen, J. (2001). An adaptive Metropolis algorithm. *Bernoulli*, *7*(2), 223–242.

Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*. https://arxiv.org/abs/http://arxiv.org/abs/1312.6114v10

McLachlan, G., & Peel, D. (2000). *Finite mixture models*. Wiley.

Nelsen, R. B. (2006). *An introduction to copulas* (2nd ed.). Springer.

Papamakarios, G., Nalisnick, E., Rezende, D. J., Mohamed, S., & Lakshminarayanan, B. (2021). Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, *22*(57), 1–64. http://jmlr.org/papers/v22/19-1028.html

Papamakarios, G., Pavlakou, T., & Murray, I. (2017). Masked autoregressive flow for density estimation. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in neural information processing systems* (Vol. 30). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/6c1da886822c67822bcf3679d04369fa-Paper.pdf

Parzen, E. (1962). On Estimation of a Probability Density Function and Mode. *The Annals of Mathematical Statistics*, *33*(3), 1065–1076. https://doi.org/10.1214/aoms/1177704472

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

R Core Team. (2023). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. https://www.R-project.org/

Rosenblatt, M. (1956). Remarks on Some Nonparametric Estimates of a Density Function. *The Annals of Mathematical Statistics*, *27*(3), 832–837. https://doi.org/10.1214/aoms/1177728190

Sastry, C. S., Lehrmann, A., Brubaker, M. A., & Radovic, A. (2022). Efficient CDF approximations for normalizing flows. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=LnjclqBl8R

Sick, B., Hathorn, T., & Durr, O. (2021). Deep transformation models: Tackling

DRAFT

complex regression problems with neural network based transformation models. *2020 25th International Conference on Pattern Recognition (ICPR)*, 2476–2481. https://doi.org/10.1109/ICPR48806.2021.9413177

Tabak, E. G., & Turner, C. V. (2013). A family of nonparametric density estimation algorithms. *Communications on Pure and Applied Mathematics*, *66*(2), 145–164. https://doi.org/https://doi.org/10.1002/cpa.21423

The Probabilists. (2024). *Zuko* (Version 1.1.0). https://zuko.readthedocs.io/stable/

Titterington, D. M., Smith, A. F. M., Smith, A. F. M., & Makov, U. E. (1985). *Statistical analysis of finite mixture distributions*. Wiley. ISBN: 9780471907633

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

DRAFT